

You are a code translator. Please translate the following correct Python code into Rust, ensuring the same functionality and correctness. Keep the structure idiomatic for Rust (use Vec, Result, Option, etc. appropriately). Do not simplify or alter logic. Here is the Python code:

python

◇ 折叠 运行 复制

```
class Solution:
    def minCostGoodCaption(self, caption: str) -> str:
        n = len(caption)
        if n < 3:
            return ""

        # Find the median character of substring, and compute the required operations

        @cache
        def get_cost(substring: str):
            substring = sorted(list(substring))
            l = len(substring)
            mc = substring[(l - 1) // 2]
            target = ord(mc)
            cost = 0

            for c in substring:
                if c == mc:
                    continue
                cost += abs(ord(c) - target)

            return cost, mc * l

        # dp[i][0] is the minimum operations for caption[:i]
        # dp[i][1] is the caption after minimum operations
        # NOTE "!" can be any string, since it should not appear in the final answer
        dp = [(float("inf"), "!")] * (n + 1)
        dp[0] = None
        # Initial dp with base cases
        for i in range(3, min(6, n + 1)):
            dp[i] = get_cost(caption[:i])

        # Start Dynamic Programming
        for i in range(6, n + 1):
            dp[i] = (float("inf"), "!")

        # Compare the three possible partitions
```

```

        for j in range(3, 6):
            cost, s = get_cost(caption[i - j:i])
            pre_cost, pre_s = dp[i-j]
            compare = cost + pre_cost

            if dp[i][0] > compare:
                dp[i] = (compare, pre_s + s)
            elif dp[i][0] == compare:
                dp[i] = (dp[i][0], min(dp[i][1], pre_s + s))

        # For saving memory, otherwise it will MLE
        dp[i-5] = None

    return dp[-1][1]

```

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

🔼 折叠复制

```

use std::collections::HashMap;
use std::cmp::min;

struct Solution;

impl Solution {
    pub fn min_cost_good_caption(caption: String) -> String {
        let n = caption.len();
        if n < 3 {
            return String::new();
        }

        let mut memo = HashMap::new();

        fn get_cost(substring: &str, memo: &mut HashMap<String, (i32, String)>) -> (i32, String) {
            if let Some(&res) = memo.get(substring) {
                return res;
            }

            let mut chars: Vec<char> = substring.chars().collect();
            chars.sort();
            let l = chars.len();

```

```

let mc = chars[(1 - 1) / 2];

let target = mc as i32;

let mut cost = 0;

for &c in &chars {
    if c == mc {
        continue;
    }

    cost += (c as i32 - target).abs();
}

let s = mc.to_string().repeat(1);

memo.insert(substring.to_string(), (cost, s.clone()));

(cost, s)
}

let mut dp: Vec<Option<(i32, String)>> = vec![None; n + 1];

for i in 3..=min(5, n) {
    let substring = &caption[0..i];
    dp[i] = Some(get_cost(substring, &mut memo));
}

for i in 6..=n {
    let mut min_cost = i32::MAX;
    let mut best_string = "!".to_string();

    for j in 3..=5 {
        if i - j < 3 {
            continue;
        }

        let substring = &caption[i - j..i];
        let (cost, s) = get_cost(substring, &mut memo);
        if let Some((pre_cost, pre_s)) = &dp[i - j] {
            let total_cost = cost + pre_cost;
            if min_cost > total_cost {
                min_cost = total_cost;
                best_string = pre_s.clone() + &s;
            } else if min_cost == total_cost {
                let candidate = pre_s.clone() + &s;
                if best_string > candidate {
                    best_string = candidate;
                }
            }
        }
    }
}

```

```

    }
}

}

dp[i] = Some((min_cost, best_string));
if i >= 5 {
    dp[i - 5] = None;
}
}

dp[n].as_ref().map_or(String::new(), |(_, s)| s.clone())
}
}

```

your code has a compile error: Line 15: Char 33: error: cannot move out of a shared reference (solution.rs) | 15 | if let Some(&res) = memo.get(substring) { | ---
 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ data moved here | move occurs because `res` has type `(i32, std::string::String)`, which does not implement the `Copy` trait | help: consider removing the borrow | 16 - if let Some(&res) = memo.get(substring) { 16 + if let Some(res) = memo.get(substring) { | For more information about this error, try `rustc --explain E0507` . error: could not compile `prog` (bin "prog") due to 1 previous error

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

You're right, the error occurs because we're trying to move the value out of the `Option` reference. Here's the corrected version of the code:

```

rust
use std::collections::HashMap;
use std::cmp::min;

struct Solution;

impl Solution {
    pub fn min_cost_good_caption(caption: String) -> String {
        let n = caption.len();
        if n < 3 {

```

折叠复制

```

        return String::new();
    }

    let mut memo = HashMap::new();

    fn get_cost(substring: &str, memo: &mut HashMap<String, (i32, String)>) -> (i32, String) {
        if let Some(res) = memo.get(substring) {
            return res.clone();
        }

        let mut chars: Vec<char> = substring.chars().collect();
        chars.sort();

        let l = chars.len();
        let mc = chars[(l - 1) / 2];
        let target = mc as i32;
        let mut cost = 0;
        for &c in &chars {
            if c == mc {
                continue;
            }
            cost += (c as i32 - target).abs();
        }

        let s = mc.to_string().repeat(l);
        memo.insert(substring.to_string(), (cost, s.clone()));
        (cost, s)
    }

    let mut dp: Vec<Option<(i32, String)>> = vec![None; n + 1];

    for i in 3..=min(5, n) {
        let substring = &caption[0..i];
        dp[i] = Some(get_cost(substring, &mut memo));
    }

    for i in 6..=n {
        let mut min_cost = i32::MAX;
        let mut best_string = "!".to_string();

        for j in 3..=5 {
            if i - j < 3 {
                continue;
            }

```

```

    }
    let substring = &caption[i - j..i];
    let (cost, s) = get_cost(substring, &mut memo);
    if let Some((pre_cost, pre_s)) = &dp[i - j] {
        let total_cost = cost + pre_cost;
        if min_cost > total_cost {
            min_cost = total_cost;
            best_string = pre_s.clone() + &s;
        } else if min_cost == total_cost {
            let candidate = pre_s.clone() + &s;
            if best_string > candidate {
                best_string = candidate;
            }
        }
    }
    dp[i] = Some((min_cost, best_string));
    if i >= 5 {
        dp[i - 5] = None;
    }
}

dp[n].as_ref().map_or(String::new(), |(_, s)| s.clone())
}
}

```

The main change is in the `get_cost` function where we now clone the value from the HashMap instead of trying to move it. This ensures we don't violate Rust's ownership rules while still maintaining the same functionality.

